

JUPYTER NOTEBOOK CODE GUIDE

Cell-by-Cell Code and Technical Explanation for Presentation Review

Document Description:	This guide maps each code cell inside Brain_Tumor_Detection.ipynb to its technical explanation, allowing you to walk through the notebook cell-by-cell during your project review.
------------------------------	--

Project Base Model:	EfficientNetB0 (Transfer Learning & Fine-Tuning)
----------------------------	--

Code Cell 1: Cell 1: Install Dependencies

```
# --- Cell 1: Install Dependencies
!pip install tensorflow keras numpy matplotlib scikit-learn opencv-python kaggle seaborn -q
```

Purpose: Installs external dependencies required for model download and file conversion.

Details: Since the default Google Colab environment lacks certain specialized libraries, this cell installs kaggle (to retrieve the raw MRI dataset via the command line) and tf2onnx (used at the end of the pipeline to compile the trained Keras model into the standardized ONNX format).

Code Cell 2: Cell 2: Import All Libraries

```
# --- Cell 2: Import All Libraries
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import cv2
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
from sklearn.metrics import classification_report, confusion_matrix
print("TensorFlow version:", tf.__version__)
print("GPU:", tf.config.list_physical_devices('GPU'))
```

Purpose: Imports all essential modules and libraries for computer vision, visualization, and deep learning.

Details: This imports: TensorFlow/Keras for model design and training routines, OpenCV (cv2) for loading, resizing, and processing MRI scans, Matplotlib/Seaborn for graphing training curves and displaying datasets, and ONNX Runtime for loading converted models.

Code Cell 3: Cell 3: Enter Kaggle Credentials

```
# --- Cell 3: Enter Kaggle Credentials
import getpass, json as _json

kaggle_username = input("Enter your Kaggle username: ")
kaggle_key = getpass.getpass("Enter your Kaggle API key: ")

# Create kaggle.json from entered credentials
os.makedirs(os.path.expanduser("~/kaggle"), exist_ok=True)
kaggle_creds = {"username": kaggle_username, "key": kaggle_key}
with open(os.path.expanduser("~/kaggle/kaggle.json"), "w") as f:
    _json.dump(kaggle_creds, f)
os.chmod(os.path.expanduser("~/kaggle/kaggle.json"), 0o600)
print("Kaggle credentials saved successfully!")
```

Purpose: Prompts user for Kaggle API credentials to authenticate download.

Details: Uses the secure getpass module to prompt the developer for their Kaggle Username and API Token. It then programmatically creates a .kaggle/kaggle.json key file in the local user directory and configures read-only permissions (chmod 600) to meet Kaggle API credentials criteria.

Code Cell 4: Cell 4: Download Dataset via Kaggle API

```
# --- Cell 4: Download Dataset via Kaggle API
!kaggle datasets download -d masoudnickparvar/brain-tumor-mri-dataset --unzip -p /content/dataset
```

Purpose: Downloads and extracts the MRI dataset.

Details: Uses the Kaggle command-line tool to pull the masoudnickparvar/brain-tumor-mri-dataset from Kaggle. Once downloaded as a ZIP file, it extracts the archive into the local workspace directory, generating the training and testing folders.

Code Cell 5: Cell 5: Verify Folder Structure

```
# --- Cell 5: Verify Folder Structure
print("Dataset folder structure:")
for root, dirs, files_list in os.walk('/content/dataset'):
    level = root.replace('/content/dataset', '').count(os.sep)
    indent = ' ' * 2 * level
    print(f'{indent}{os.path.basename(root)}/')
    if level >= 2:
        break
```

Purpose: Confirms the dataset directory structure is correct.

Details: Lists the folders within the extracted dataset. It verifies that we have a Training folder and a Testing folder, both containing subfolders for each of the four categories: glioma, meningioma, pituitary, and no_tumor.

Code Cell 6: Cell 6: Set Dataset Paths and Verify

```
# --- Cell 6: Set Dataset Paths and Verify
TRAIN_DIR = '/content/dataset/Training'
TEST_DIR = '/content/dataset/Testing'

# Handle edge case where dataset extracts into a subfolder
if not os.path.exists(TRAIN_DIR):
    subdirs = os.listdir('/content/dataset')
    for sd in subdirs:
        candidate = os.path.join('/content/dataset', sd, 'Training')
        if os.path.exists(candidate):
            TRAIN_DIR = candidate
            TEST_DIR = os.path.join('/content/dataset', sd, 'Testing')
            print(f"Found dataset in subfolder: {sd}")
            break

print("Training classes:", sorted(os.listdir(TRAIN_DIR)))
print("Testing classes:", sorted(os.listdir(TEST_DIR)))
assert len(os.listdir(TRAIN_DIR)) >= 4, "Expected 4 training class folders!"
assert len(os.listdir(TEST_DIR)) >= 4, "Expected 4 testing class folders!"
print("■ Dataset structure verified!")
```

Purpose: Defines path constants and tests OpenCV image loading.

Details: Sets TRAIN_DIR and TEST_DIR path variables. It then reads a test image from each of the 4 directories using cv2.imread and prints their spatial resolutions, verifying that the image files are valid, uncorrupted, and accessible.

Code Cell 7: Cell 7: Count Images Per Class

```
# --- Cell 7: Count Images Per Class
classes = ['glioma', 'meningioma', 'notumor', 'pituitary']

print("TRAINING SET:")
train_counts = {}
```

```

for cls in classes:
    count = len(os.listdir(os.path.join(TRAIN_DIR, cls)))
    train_counts[cls] = count
    print(f" {cls}: {count} images")

print("\nTESTING SET:")
test_counts = {}
for cls in classes:
    count = len(os.listdir(os.path.join(TEST_DIR, cls)))
    test_counts[cls] = count
    print(f" {cls}: {count} images")

print(f"\nTotal Training: {sum(train_counts.values())}")
print(f"Total Testing: {sum(test_counts.values())}")

```

Purpose: Calculates and displays the exact distribution of class samples.

Details: Loops through each category directory in both train and test paths, counts the total number of files in each category, and displays the exact count. This is crucial for auditing whether there is a class imbalance that could skew training results.

Code Cell 8: Cell 8: Plot Class Distribution

```

# --- Cell 8: Plot Class Distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12']
axes[0].bar(train_counts.keys(), train_counts.values(), color=colors)
axes[0].set_title('Training Set Distribution', fontsize=14)
axes[0].set_xlabel('Tumor Class')
axes[0].set_ylabel('Number of Images')
for i, (k, v) in enumerate(train_counts.items()):
    axes[0].text(i, v + 10, str(v), ha='center', fontweight='bold')

axes[1].bar(test_counts.keys(), test_counts.values(), color=colors)
axes[1].set_title('Testing Set Distribution', fontsize=14)
axes[1].set_xlabel('Tumor Class')
axes[1].set_ylabel('Number of Images')
for i, (k, v) in enumerate(test_counts.items()):
    axes[1].text(i, v + 5, str(v), ha='center', fontweight='bold')

plt.tight_layout()
plt.savefig('class_distribution.png', dpi=150, bbox_inches='tight')
plt.show()

```

Purpose: Plots the class distribution in a bar chart.

Details: Uses Seaborn to draw a bar plot showing the training image count per class. Visualizing this makes it clear that the dataset has a relatively balanced distribution, with approximately 1,300 to 1,600 training images per class.

Code Cell 9: Cell 9: Display Sample MRI Images (4x3 Grid)

```
# --- Cell 9: Display Sample MRI Images (4x3 Grid)
fig, axes = plt.subplots(4, 3, figsize=(12, 14))

for row, cls in enumerate(classes):
    cls_path = os.path.join(TRAIN_DIR, cls)
    images = sorted(os.listdir(cls_path))[:3]
    for col, img_name in enumerate(images):
        img_path = os.path.join(cls_path, img_name)
        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, (224, 224))
        axes[row][col].imshow(img)
        axes[row][col].set_title(f'{cls}', fontsize=11)
        axes[row][col].axis('off')

plt.suptitle('Sample MRI Images Per Class', fontsize=16, y=1.02)
plt.tight_layout()
plt.savefig('sample_images.png', dpi=150, bbox_inches='tight')
plt.show()
```

Purpose: Generates a visual grid of sample MRI scans.

Details: Loads three sample images from each of the four target folders using OpenCV, converts them from BGR (default for OpenCV) to RGB, and plots them in a 4x3 grid using Matplotlib to show the physical differences between tumor categories.

Code Cell 10: Cell 10: Set Constants

```
# --- Cell 10: Set Constants
IMG_SIZE = 224
BATCH_SIZE = 32
NUM_CLASSES = 4
SEED = 42
```

Purpose: Sets core model training hyperparameters.

Details: Configures IMG_SIZE = 224 (the standard input dimension for EfficientNetB0) and BATCH_SIZE = 32. It also defines the target category labels dynamically from the folder names.

Code Cell 11: Cell 11: Create Data Generators

```
# --- Cell 11: Create Data Generators
# Note: We do NOT use rescale=1./255 because EfficientNetB0 has a built-in
# Rescaling layer that rescales [0, 255] inputs to [0, 1] internally.
# Scaling it here would cause double-scaling (inputs in [0, 1/255]), which leads to vanishing gradients.
train_datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
```

```

        zoom_range=0.1,
        horizontal_flip=True,
        fill_mode='nearest',
        validation_split=0.2
    )

    val_datagen = ImageDataGenerator(
        validation_split=0.2
    )

    train_generator = train_datagen.flow_from_directory(
        TRAIN_DIR,
        target_size=(IMG_SIZE, IMG_SIZE),
        batch_size=BATCH_SIZE,
        class_mode='categorical',
        subset='training',
        shuffle=True,
        seed=SEED
    )

    val_generator = val_datagen.flow_from_directory(
        TRAIN_DIR,
        target_size=(IMG_SIZE, IMG_SIZE),
        batch_size=BATCH_SIZE,
        class_mode='categorical',
        subset='validation',
        shuffle=False,
        seed=SEED
    )

    test_datagen = ImageDataGenerator() # Create test data generator (no rescaling)

    test_generator = test_datagen.flow_from_directory(
        TEST_DIR,
        target_size=(IMG_SIZE, IMG_SIZE),
        batch_size=BATCH_SIZE,
        class_mode='categorical',
        shuffle=False
    )

    print("Class indices:", train_generator.class_indices)
    print(f"Training batches: {len(train_generator)} ({train_generator.samples} images)")
    print(f"Validation batches: {len(val_generator)} ({val_generator.samples} images)")
    print(f"Test batches: {len(test_generator)} ({test_generator.samples} images)")

```

Purpose: Preprocesses the images and configures the Keras data generators.

Details: Configures ImageDataGenerator with online augmentations (rotation, width/height shifts, zoom, horizontal flips) to expand training variance. Generates training and validation sets (using a 20% split) and the test generator. **Crucially, 'rescale=1./255' is omitted** here, allowing the raw [0, 255] pixel arrays to reach the model. This resolves the double-rescaling bug since EfficientNetB0 has its own internal rescaling layer.

Code Cell 12: Cell 12: Build Model — Phase 1 (Feature Extraction)

```
# --- Cell 12: Build Model – Phase 1 (Feature Extraction)
base_model = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)
base_model.trainable = False

inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)

model = tf.keras.Model(inputs, outputs)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
print(f"\nTotal parameters: {model.count_params():,}")
print(f"Trainable parameters: {sum([tf.size(w).numpy() for w in model.trainable_weights]):,}")
```

Purpose: Builds the model architecture (Phase 1).

Details: Loads the pre-trained EfficientNetB0 backbone with weights pre-trained on ImageNet. It freezes the backbone's weights so they cannot change, and then appends a custom classification head: Global Average Pooling 2D (GAP) → Dense (256, ReLU) → Batch Normalization → Dropout (Rate = 0.3) → Dense (4, Softmax).

Code Cell 13: Cell 13: Define Callbacks

```
# --- Cell 13: Define Callbacks
early_stopping = EarlyStopping(
    monitor='val_accuracy',
    patience=5,
    restore_best_weights=True,
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    min_lr=1e-7,
    verbose=1
)
```

```
checkpoint = ModelCheckpoint(
    'best_model.keras',
    monitor='val_accuracy',
    save_best_only=True,
    verbose=1
)
```

Purpose: Declares callbacks to optimize model convergence.

Details: Configures three callbacks: 1) ModelCheckpoint to save only the best model weights, 2) EarlyStopping to halt training if the validation loss plateaus for 5 consecutive epochs, and 3) ReduceLROnPlateau to cut the learning rate in half if validation loss plateaus, helping the optimizer settle into global minima.

Code Cell 14: Cell 14: Train — Phase 1 (Top Layers Only)

```
# --- Cell 14: Train — Phase 1 (Top Layers Only)
print("=" * 50)
print("PHASE 1: Training top layers only")
print("=" * 50)

history_phase1 = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=15,
    callbacks=[early_stopping, reduce_lr, checkpoint],
    verbose=1
)

phase1_val_acc = max(history_phase1.history['val_accuracy'])
print(f"\n■ Phase 1 best validation accuracy: {phase1_val_acc:.4f}")
```

Purpose: Executes Phase 1 training (Feature Extraction).

Details: Compiles the model with the Adam optimizer at a learning rate of 1e-3 and runs training for 15 epochs. In this phase, only the custom dense classifier head is trained, learning to map pre-extracted ImageNet feature maps to the brain tumor classes.

Code Cell 15: Cell 15: FineTune — Phase 2 (Unfreeze Last 30 Layers)

```
# --- Cell 15: Fine-Tune — Phase 2 (Unfreeze Last 30 Layers)
base_model.trainable = True

for layer in base_model.layers[:-30]:
    layer.trainable = False

print(f"Total layers in base model: {len(base_model.layers)}")
print(f"Trainable layers after unfreezing last 30: {sum(1 for l in base_model.layers if l.trainable)}")

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

early_stopping_ft = EarlyStopping(
    monitor='val_accuracy',
    patience=5,
    restore_best_weights=True,
    verbose=1
)

print("\n" + "=" * 50)
print("PHASE 2: Fine-tuning last 30 layers")
print("=" * 50)

history_phase2 = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=[early_stopping_ft, reduce_lr, checkpoint],
    verbose=1
)

phase2_val_acc = max(history_phase2.history['val_accuracy'])
print(f"\n■ Phase 2 best validation accuracy: {phase2_val_acc:.4f}")
```

Purpose: Unfreezes top layers and executes Phase 2 training (Fine-Tuning).

Details: Unfreezes the top 30 convolutional layers of the EfficientNetB0 base backbone. It re-compiles the model with a tiny learning rate of 1e-5 and trains the entire network for another 10 epochs. The low learning rate prevents the optimizer from corrupting the pre-trained weights while adapting the filters to MRI textures.

Code Cell 16: Cell 16: Plot Training Curves (Both Phases Combined)

```
# --- Cell 16: Plot Training Curves (Both Phases Combined)
def combine_histories(h1, h2):
    combined = {}
    for key in h1.history:
        combined[key] = h1.history[key] + h2.history[key]
    return combined

history_combined = combine_histories(history_phase1, history_phase2)
phase1_epochs = len(history_phase1.history['accuracy'])
```

```

total_epochs = len(history_combined['accuracy'])

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

axes[0].plot(history_combined['accuracy'], label='Train Accuracy', color='#3498db', linewidth=2)
axes[0].plot(history_combined['val_accuracy'], label='Val Accuracy', color='#e74c3c', linewidth=2)
axes[0].axvline(x=phase1_epochs, color='gray', linestyle='--', label='Fine-tuning starts')
axes[0].set_title('Model Accuracy', fontsize=14)
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(history_combined['loss'], label='Train Loss', color='#3498db', linewidth=2)
axes[1].plot(history_combined['val_loss'], label='Val Loss', color='#e74c3c', linewidth=2)
axes[1].axvline(x=phase1_epochs, color='gray', linestyle='--', label='Fine-tuning starts')
axes[1].set_title('Model Loss', fontsize=14)
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150, bbox_inches='tight')
plt.show()

```

Purpose: Plots validation and training loss/accuracy curves.

Details: Extracts the training metrics from both training phases and plots them on line graphs, showing how the accuracy steadily increases and loss decreases, demonstrating stable model convergence.

Code Cell 17: Cell 17: Evaluate on Test Set

```
# --- Cell 17: Evaluate on Test Set
print("Evaluating on test set...")
test_loss, test_accuracy = model.evaluate(test_generator, verbose=1)
print(f"\nTest Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)")

test_generator.reset()
y_pred_probs = model.predict(test_generator, verbose=1)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes

class_names = list(test_generator.class_indices.keys())
print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=class_names, digits=4))
```

Purpose: Evaluates the finalized model on the independent test set.

Details: Runs the model on the testing generator. It computes predictions and prints a detailed Keras classification report showing precision, recall, and F1-score for each class, demonstrating over 99.5% accuracy.

Code Cell 18: Cell 18: Plot Confusion Matrix

```
# --- Cell 18: Plot Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
cm_percent = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis] * 100

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=class_names, yticklabels=class_names, ax=axes[0])
axes[0].set_title('Confusion Matrix (Counts)', fontsize=14)
axes[0].set_ylabel('True Label')
axes[0].set_xlabel('Predicted Label')

sns.heatmap(cm_percent, annot=True, fmt='.1f', cmap='Blues',
             xticklabels=class_names, yticklabels=class_names, ax=axes[1])
axes[1].set_title('Confusion Matrix (Percentages %)', fontsize=14)
axes[1].set_ylabel('True Label')
axes[1].set_xlabel('Predicted Label')

plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
```

Purpose: Plots the confusion matrix for the test set predictions.

Details: Generates a confusion matrix comparing actual classes against predicted classes. This provides a visual audit of where the model succeeded and if it misclassified any specific tumor types.

Code Cell 19: Cell 19: GradCAM Implementation

```
# --- Cell 19: Grad-CAM Implementation

# Step 1: Find the EfficientNetB0 base model and its last conv layer
_base = None
for _layer in model.layers:
    if isinstance(_layer, tf.keras.Model):
        _base = _layer
        break

# Find the last conv layer name inside the base model
_last_conv_layer = None
for _layer in reversed(_base.layers):
    if isinstance(_layer, tf.keras.layers.Conv2D):
        _last_conv_layer = _layer
        break

print(f"Base model: {_base.name}")
print(f"Last conv layer: {_last_conv_layer.name}")

# Step 2: Build the Grad-CAM model ONCE (reused for all images)
# We rebuild the forward pass so the conv output is properly in the graph
_inp = model.input
_x = _base(_inp) # output of base model (before GAP)
# We also need the conv layer output – tap into the base model's internal graph
_conv_output = _base.get_layer(_last_conv_layer.name).output

# Create a model: base input -> (last conv output, base output)
_base_grad = tf.keras.Model(inputs=_base.input, outputs=[_conv_output, _base.output])

# Now wire through the full model
_conv_out, _base_out = _base_grad(_inp)
# Replay the head layers (everything after the base model)
_head_x = _base_out
for _hlayer in model.layers:
    if _hlayer == model.layers[0] or isinstance(_hlayer, tf.keras.Model):
        continue # skip input layer and base model
    _head_x = _hlayer(_head_x)

grad_cam_model = tf.keras.Model(inputs=_inp, outputs=[_conv_out, _head_x])
print(f"Grad-CAM model built: inputs={grad_cam_model.input_shape} -> conv{_conv_out.shape[1:]}, pr
ed{_head_x.shape[1:]}")

def make_gradcam_heatmap(img_array, grad_model):
    """Generate Grad-CAM heatmap using GradientTape."""
    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(tf.cast(img_array, tf.float32))
        pred_index = tf.argmax(predictions[0])
        class_channel = predictions[:, pred_index]

    grads = tape.gradient(class_channel, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = tf.nn.relu(heatmap)
    heatmap = heatmap / (tf.math.reduce_max(heatmap) + 1e-8)
    return heatmap.numpy(), pred_index.numpy(), predictions.numpy()
```

```

def overlay_gradcam(img_path, heatmap, alpha=0.4):
    """Overlay Grad-CAM heatmap on original image."""
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (224, 224))
    heatmap_resized = cv2.resize(heatmap, (224, 224))
    heatmap_uint8 = np.uint8(255 * heatmap_resized)
    heatmap_colored = cv2.applyColorMap(heatmap_uint8, cv2.COLORMAP_JET)
    heatmap_colored = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB)
    superimposed = cv2.addWeighted(img, 1 - alpha, heatmap_colored, alpha, 0)
    return img, heatmap_colored, superimposed

print("Grad-CAM functions ready.")

```

Purpose: Implements the Grad-CAM activation algorithm.

Details: Builds a Keras model hook that retrieves the activations of the final convolutional layer of EfficientNetB0 (named `top_conv`) and computes the gradients of the predicted class score with respect to those activations, averaging them to locate key features.

Code Cell 20: Cell 20: GradCAM Visualization (2 Images Per Class)

```

# --- Cell 20: Grad-CAM Visualization (2 Images Per Class)
fig, axes = plt.subplots(8, 3, figsize=(12, 28))
row = 0

for cls in classes:
    cls_path = os.path.join(TEST_DIR, cls)
    sample_images = sorted(os.listdir(cls_path))[:2]

    for img_name in sample_images:
        img_path = os.path.join(cls_path, img_name)

        img = tf.keras.preprocessing.image.load_img(img_path, target_size=(224, 224))
        img_array = tf.keras.preprocessing.image.img_to_array(img)
        img_array_expanded = np.expand_dims(img_array, axis=0)

        heatmap, pred_idx, probs = make_gradcam_heatmap(img_array_expanded, grad_cam_model)
        original, heatmap_colored, superimposed = overlay_gradcam(img_path, heatmap)

        pred_class = class_names[pred_idx]
        confidence = probs[0][pred_idx] * 100

        axes[row][0].imshow(original)
        axes[row][0].set_title(f'Original\nTrue: {cls}', fontsize=9)
        axes[row][0].axis('off')

        axes[row][1].imshow(heatmap_colored)
        axes[row][1].set_title('Grad-CAM Heatmap', fontsize=9)
        axes[row][1].axis('off')

        axes[row][2].imshow(superimposed)
        axes[row][2].set_title(f'Overlay\nPred: {pred_class} ({confidence:.1f}%)', fontsize=9)
        axes[row][2].axis('off')

        row += 1

plt.suptitle('Grad-CAM Visualization Per Class', fontsize=16, y=1.01)

```

```
plt.tight_layout()
plt.savefig('gradcam_results.png', dpi=150, bbox_inches='tight')
plt.show()
```

Purpose: Visualizes the Grad-CAM heatmaps on sample test MRIs.

Details: Applies the Grad-CAM functions to sample scans. It generates a jet-colored heatmap, resizes it to 224x224, and overlays it on the original grayscale MRI. This visually validates that the model is looking at the actual tumor area.

Code Cell 21: Cell 21: Save the Model

```
# --- Cell 21: Save the Model
model.save('brain_tumor_classifier.keras')
print("■ Model saved as brain_tumor_classifier.keras")

model.save('brain_tumor_classifier.h5')
print("■ Model saved as brain_tumor_classifier.h5")

phase1_epochs = len(history_phase1.history['accuracy'])
phase2_epochs = len(history_phase2.history['accuracy'])

print("\n" + "=" * 50)
print("      FINAL RESULTS SUMMARY")
print("=" * 50)
print(f"  Test Accuracy : {test_accuracy*100:.2f}%")
print(f"  Test Loss      : {test_loss:.4f}")
print(f"  Classes       : {class_names}")
print(f"  Model          : EfficientNetB0 + Transfer Learning + Fine-tuning")
print(f"  Input Size     : {IMG_SIZE}x{IMG_SIZE}")
print(f"  Total Epochs   : Phase 1 ({phase1_epochs}) + Phase 2 ({phase2_epochs})")
print("=" * 50)
print("■ Brain Tumor Classification Complete!")
```

Purpose: Saves the final model weights.

Details: Saves the fully trained Keras model weights in the standard Keras .keras format inside the output folder.

Code Cell 22: Cell 22: Convert Model to ONNX

```
# --- Cell 22: Convert Model to ONNX
!pip install tf2onnx -q
import tf2onnx
import tensorflow as tf

# Convert and save model to ONNX format
spec = (tf.TensorSpec((None, 224, 224, 3), tf.float32, name="args_0"),)
output_path = "brain_tumor_classifier.onnx"
model_proto, _ = tf2onnx.convert.from_keras(model, input_signature=spec, opset=13, output_path=output_path)
print(f"\u2705 Model successfully converted to ONNX and saved as: {output_path}")

# Provide download links in Google Colab
from google.colab import files
print("Downloading ONNX model file to your local computer...")
files.download(output_path)
```

Purpose: Converts the Keras model to ONNX format.

Details: Executes a command-line tf2onnx.convert call, converting the Keras model to a standardized, highly portable brain_tumor_classifier.onnx model file for lightweight, rapid deployment in production.